Supervyse OPF MetricReport and Triggers User Guide

Revision 1.0
Dec. 2024

Copyright (c) 2024, All Rights Reserved. Insyde Software.

No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form, or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of Insyde Software Corp.

Disclaimer

Insyde Software provides this document and the programs "as is" without warranty of any kind, either expressed or implied, including, but not limited to, the implied warranties of merchantability or fitness for a particular purpose.

This document could contain technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in future revisions of this document. Insyde Software is under no obligation to notify any person of the changes.

The following trademarks are used in this document:

Insyde and InsydeH2O are registered trademarks or trademarks of Insyde Software. All other trademarks or trade names are property of their respective holders.

Contents

5
1.15
1.25
2 MetricReportDefinitions and MetricReports**Error! Bookmark not defined.**
2.1 Description.....6
3 TriggersCollection 8
3.1 Description.....8
4 Reference..... 10

Revision History

Revision Number	Description	Author	Reviewer	Release Date
1.0	Initial release.	Kevin WM Chen	Victor Wu	2025-02-19

1 Introduction

This document aims to assist developers, users, and testers in understanding the specifications and usage of MetricReportDefinitions and Triggers.

1.1 Overview

The purpose of this document is to introduce in detail the design specifications of MetricReportDefinitions and Triggers, and how to correctly use these operations for system management.

1.2 Related Information

The following publications and sources of information are referred to by this document or may be useful to you:

- *Redfish Telemetry White Paper DSP2051, DMTF*

2 MetricReportDefinitions and MetricReports

2.1 Description

In BMC, there are many sensors in many different chassis, we can use MetricReportDefinitions to monitor those we are interested in.

We have the situation as below and we will show each step to make a MetricReportDefinition.

- We can find Chassis members in ChassisCollection, for example, R4000_Chassis is the ChassisId we want.

```
* Redfish URI: /redfish/v1/Chassis

* HTTP Method: GET
```

- In R4000_Chassis Sensors (/redfish/v1/Chassis/R4000_Chassis/Sensors).

```
* Redfish URI: /redfish/v1/Chassis/R4000_Chassis

* HTTP Method: GET
```

- In those sensors, we take airflow_System_Airflow as example.

```
* Redfish URI: /redfish/v1/Chassis/R4000_Chassis/Sensors/airflow_System_Airflow

* HTTP Method: GET
```

- We can monitor the sensor airflow_System_Airflow by creating a member named MyMetric in MetricReportDefinitionsCollection.

```
* Redfish URI: /redfish/v1/TelemetryService/MetricReportDefinitions

* HTTP Method: POST

* Payload:

{

    "Id":"MyMetric",

    "Name": "MyMRD",

    "Metrics":[

        {

            "MetricProperties": [

                "/redfish/v1/Chassis/R4000_Chassis/Sensors/airflow_System_Airflow"

            ]

        }

    ],
```

```
    "MetricReportDefinitionType":"Periodic",

    "ReportActions":["RedfishEvent"],

    "Schedule":{"RecurrenceInterval":"PT1S"}

}
```

- After creating a MetricReportDefinitions MyMetric, a member of MetricReportCollection which has the same name with MetricReportDefinitions will be generated, we can check the reading value of sensor arflow_System_Airflow in the new generated MetricReports MyMetric.

```
* Redfish URI: /redfish/v1/TelemetryService/MetricReports/MyMetric

* HTTP Method: GET
```

3 TriggersCollection

3.1 Description

Within the example of MetricReportDefinitions and MetricReport, if we not only want to get the reading value but also demand it to take some actions such as sending an Event if the reading value reached the threshold, we can use Triggers for the sensor.

- We can find Chassis members in ChassisCollection (/redfish/v1/Chassis), for example, R4000_Chassis is the ChassisId we want.

```
* Redfish URI: /redfish/v1/Chassis

* HTTP Method: GET
```

- In R4000_Chassis Sensors (/redfish/v1/Chassis/R4000_Chassis/Sensors)

```
* Redfish URI: /redfish/v1/Chassis/R4000_Chassis

* HTTP Method: GET
```

- In those sensors, we take airflow_System_Airflow as example.

```
* Redfish URI: /redfish/v1/Chassis/R4000_Chassis/Sensors/airflow_System_Airflow

* HTTP Method: GET
```

- We can create a member in TriggersCollection by using the following command

```
* Redfish URI: /redfish/v1/TelemetryService/Triggers

* HTTP Method: POST

* Payload:

{

    "Id": "MyTrigger1",

    "Name": "MyT1",

    "MetricType": "Numeric",

    "TriggerActions": ["RedfishEvent"],

    "NumericThresholds":

        {

            "UpperCritical":

                {

                    "Reading": 9.5,

                    "Activation": "Increasing",

                    "DwellTime": "PT0.001S"
```

```
                },
            "UpperWarning":
                {
                    "Reading": 9.0,
                    "Activation": "Increasing",
                    "DwellTime": "PT0.004S"
                }
            },
        "MetricProperties":
        [
            "/redfish/v1/Chassis/R4000_Chassis/Sensors/airflow_System_Airflow"
        ],
        "Links": {"MetricReportDefinitions": []}
}
```

4 Reference

Redfish Telemetry White Paper DSP2051, DMTF